

RESOLUÇÃO DE 10 EXERCÍCIOS

NOME: Fernando Da Silva Dala

Matrícula: 2610100511

5. Gerador de tabela ASCII

python

```
print("Tabela ASCII (32 a 126):")
print("-" * 40)
print("Código | Caractere | Tipo")
print("-" * 40)

for codigo in range(32, 127):
    caractere = chr(codigo)

    if caractere.isalpha():
        tipo = "Letra"
    elif caractere.isdigit():
        tipo = "Dígito"
    else:
        tipo = "Símbolo"

    print(f"{codigo:6d} | {caractere:^9} | {tipo}")
```

6. Dimensionamento de cabo elétrico

python

```
corrente = float(input("Digite a corrente elétrica (A): "))

if corrente <= 10:
    secao = "1.5 mm²"
elif corrente <= 16:
    secao = "2.5 mm²"
elif corrente <= 25:
    secao = "4 mm²"
elif corrente <= 35:
    secao = "6 mm²"
```

```

else:
    secao = "consultar engenheiro"

    print(f"\nPara {corrente}A, a seção recomendada é: {secao}")

```

7. Somador de séries matemáticas

```

python

import math

x = float(input("Digite o valor de x: "))
N = int(input("Digite o número de termos N: "))

# Série e^x
ex = 0
for n in range(N):
    ex += (x ** n) / math.factorial(n)

# Série sin(x)
sin_x = 0
for n in range(N):
    sin_x += ((-1) ** n) * (x ** (2*n + 1)) / math.factorial(2*n + 1)

print(f"\ne^{x} ≈ {ex:.6f}")
print(f"sin({x}) ≈ {sin_x:.6f}")
print(f"Valor real e^{x}: {math.exp(x):.6f}")

    print(f"Valor real sin({x}): {math.sin(x):.6f}")

```

8. Sistema de média escolar

```

python

N = int(input("Quantos alunos? "))
medias = []

for i in range(N):
    nome = input(f"\nNome do {i+1}º aluno: ")
    notas = []

    for j in range(4):

```

```

        nota = float(input(f"Nota {j+1}: "))
        notas.append(nota)

media = sum(notas) / 4
medias.append(media)

if media >= 7:
    situacao = "Aprovado"
elif media >= 5:
    situacao = "Recuperação"
else:
    situacao = "Reprovado"

print(f"{nome}: média {media:.1f} - {situacao}")

media_turma = sum(medias) / N

print(f"\nMédia da turma: {media_turma:.1f}")

```

9. Simulação de ensaio de compressão

```

python

forca = 0
deformacao = 0
falhou = False

while not falhou:
    forca += 100
    deformacao += forca * 0.0001

    print(f"Força: {forca} N | Deformação: {deformacao:.2f} mm")

    if deformacao > 5:
        falhou = True
        print(f"\nPeça falhou com força de {forca} N")

        print(f"Deformação final: {deformacao:.2f} mm")

```

10. Crivo de Eratóstenes

```
python
```

```

N = int(input("Digite o valor limite N: "))
if N < 2:
    print("Não há números primos até este valor.")
else:
    # Inicializa lista de booleanos (True = primo)
    primo = [True] * (N + 1)
    primo[0] = primo[1] = False

    # Crivo de Eratóstenes
    for i in range(2, int(N ** 0.5) + 1):
        if primo[i]:
            for j in range(i * i, N + 1, i):
                primo[j] = False

    # Exibe resultados
    primos = [i for i in range(2, N + 1) if primo[i]]
    print(f"\nNúmeros primos até {N}:")
    print(primos)

    print(f"Total encontrado: {len(primos)}")

```

11. Calculadora de ponto de equilíbrio

```

python

custo_fixo = float(input("Custo fixo (R$): "))
custo_variavel = float(input("Custo variável por unidade (R$): "))
preco_venda = float(input("Preço de venda por unidade (R$): ")) N
= int(input("Número máximo de unidades: "))

ponto_equilibrio = None

print("\nUnidades | Lucro/Prejuízo | Situação")
print("-" * 45)

for unidades in range(0, N + 1):
    lucro = (preco_venda - custo_variavel) * unidades - custo_fixo

    if lucro < 0:
        situacao = "PREJUÍZO"
    elif lucro == 0:
        situacao = "EQUILÍBRIO"
    if ponto_equilibrio is None:

```

```

        ponto_equilibrio = unidades
    else:
        situacao = "LUCRO"
        print(f"{unidades:8d} | R$ {lucro:10.2f} | {situacao}")

if ponto_equilibrio is not None:
    print(f"\nPonto de equilíbrio: {ponto_equilibrio} unidades")
else:
    print(f"\nPonto de equilíbrio: {custo_fixo / (preco_venda -
custo_variavel):.0f} unidades")

```

12. Análise de pontos

```

python

import math

N = int(input("Quantos pontos? "))
pontos = []

# Leitura dos pontos
for i in range(N):
    print(f"Ponto {i+1}:")
    x = float(input(" x: "))
    y = float(input(" y: "))
    pontos.append((x, y))

# Cálculos
distancias = []
maior_dist_origem = 0
ponto_mais_distante = None

for i, (x, y) in enumerate(pontos):
    # Distância da origem
    dist_origem = math.sqrt(x**2 + y**2)
    if dist_origem > maior_dist_origem:
        maior_dist_origem = dist_origem
        ponto_mais_distante = (x, y)

# Distância entre pontos consecutivos
for i in range(N):
    x1, y1 = pontos[i]

```

```

x2, y2 = pontos[(i + 1) % N] # Último com primeiro
dist = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
distancias.append(dist)
dist_media = sum(distancias) / N
perimetro = sum(distancias)

print(f"\nDistância média entre pontos consecutivos: {dist_media:.2f}")
print(f"Ponto mais distante da origem: {ponto_mais_distante} (distância: {maior_dist_origem:.2f})")

print(f"Perímetro total do polígono: {perimetro:.2f}")

```

13. Simulação de linha de transmissão

```

python

tensao_inicial = float(input("Tensão inicial (V): "))
distancia_total = float(input("Distância total (km): "))

tensao = tensao_inicial

print(f"\nDistância (km) | Tensão (V) | Status")
print("-" * 45)

for km in range(1, int(distancia_total) + 1):
    tensao *= 0.995 # Redução de 0.5%

    if tensao < tensao_inicial * 0.90:
        status = "CRÍTICO (<90%)"
    elif tensao < tensao_inicial * 0.95:
        status = "ALERTA (<95%)"
    else:
        status = "Normal"

    print(f"{km:13d} | {tensao:9.2f} | {status}")

print(f"\nTensão final: {tensao:.2f} V")

print(f"Perda total: {(1 - tensao/tensao_inicial)*100:.1f}%")

```

14. Controle de estoque

python

```
N = int(input("Quantos movimentos? "))
saldo = 0
movimentos = []

for i in range(N):
    print(f"\nMovimento {i+1}:")
    tipo = input("Tipo (entrada/saida): ").lower()
    quantidade = float(input("Quantidade: "))

    if tipo == "entrada":
        saldo += quantidade
        movimentos.append(("ENTRADA", quantidade))
    elif tipo == "saida":
        if quantidade > saldo:
            print("ERRO: Quantidade maior que o saldo!")
            continue
        saldo -= quantidade
        movimentos.append(("SAÍDA", quantidade))
    else:
        print("Tipo inválido!")
        continue

print(f"Saldo atual: {saldo:.0f}")

if saldo < 10:
    print("ALERTA: Estoque abaixo de 10 unidades!")

print("\nRELATÓRIO FINAL")
print("-" * 30)
for i, (tipo, qtd) in enumerate(movimentos):
    print(f"{i+1:2d}. {tipo:7s}: {qtd:6.0f}")
print("-" * 30)

print(f"Saldo final: {saldo:.0f} unidades")
```

15. Simulador de antena Yagi

python

```
print("Ganho de antena Yagi")
print("-" * 40)
```

```
print("Elementos | Ganho (dBi)")
print("-" * 40)

for n in range(2, 21):
    G = 10 * (n ** 0.8) # Aproximação:  $10 * \log_{10}(0.8 * n)$ 
    print(f"{n:9d} | {G:11.2f}")
print("-" * 40)

print("Quanto mais elementos, maior o ganho!")
```

Estes programas implementam todas as funcionalidades solicitadas, utilizando estruturas de controle como `for`, `while`, `if/elif/else`, listas e funções matemáticas conforme necessário.